Technical Report - **Product specification**

EasySpot

Course: IES - Introdução à Engenharia de Software

Date: Aveiro, 22/02/2026

Students: 127368: Claudino Martins
125895: Inês Lourenço
125102: Maria Mané
124833: Martim Gil

Project Abstract: EasySpot is a real-time parking analytics platform that shows live parking occupancy, EV charger status, and accessible-bay availability across urban sites.

Table of Contents:

1. Introduction

EasySpot is a parking analytics app that provides real-time parking occupancy, EV charger status, and accessible bay availability. Drivers can quickly view available bays and fixed hourly rates on the app. Operators access read-only dashboards with occupancy and revenue reports, while technicians monitor sensor health, diagnostics, and event logs.

2. Product concept

Vision Statement

Concept

EasySpot, is a smart parking management platform that provides real-time location and monitoring of parking spaces. It offers a platform across web and mobile interfaces for all users: a driver experience to find, filter, and navigate to available spots, and a data dashboard for operators. The system integrates IoT sensor APIs to capture occupancy data, generate usage statistics, and provide predictive insights.

Inspiration

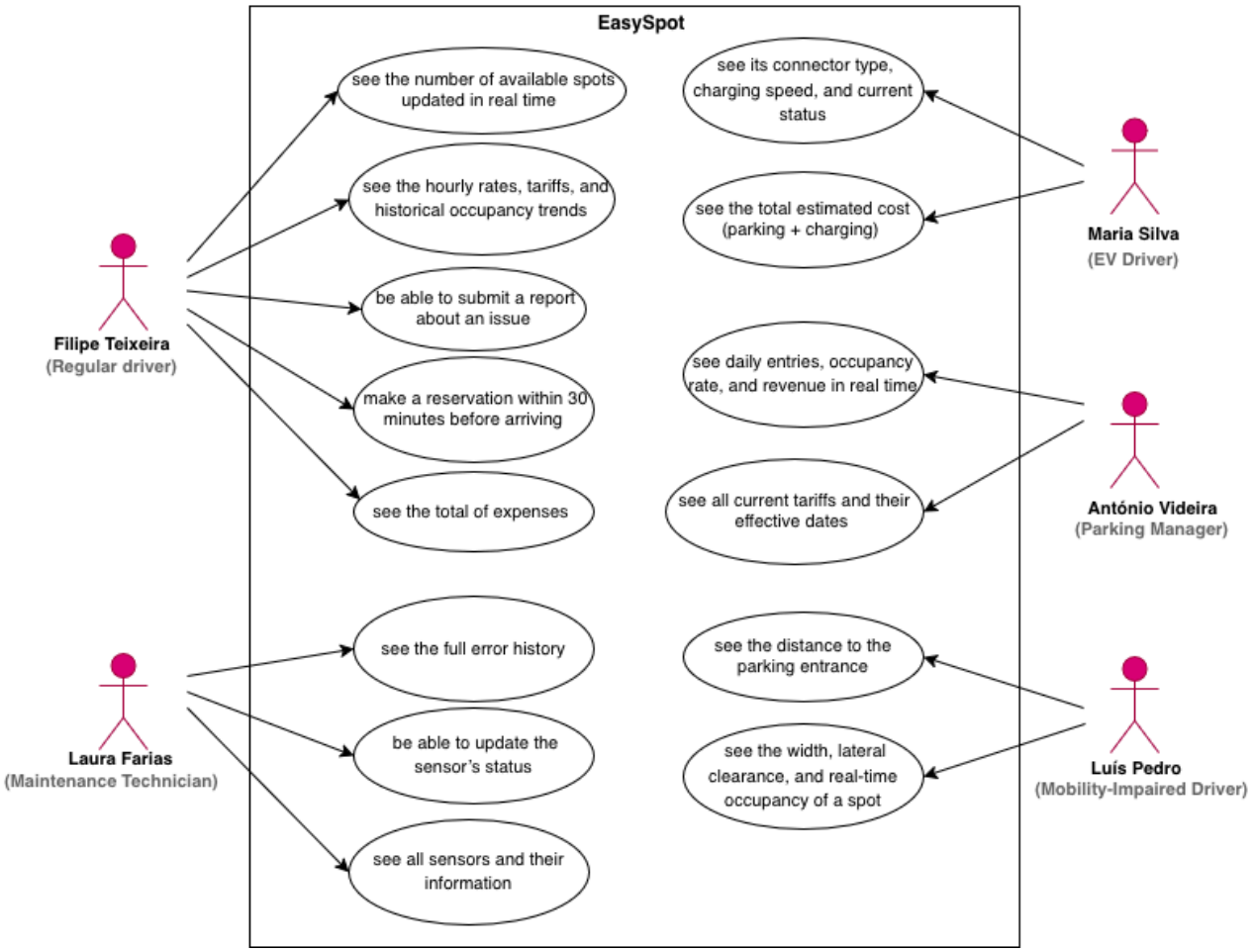
While there are existing navigation tools like Google Maps or Waze that offer basic location services, they often lack granular, real-time data regarding specific parking availability and specialized needs (such as EV charging status or accessibility features). We were inspired by the growing frustration of "circling the block" in urban centers, which wastes time. Unlike traditional systems, EasySpot differentiates itself by offering routing for electric vehicles and disabled drivers, alongside a management API that allows parking owners to monitor financial returns and hardware health in one place.

Motivation

The motivation behind EasySpot comes from the need to optimize urban space and improve the daily lives of diverse user groups. We wanted to solve the "last-mile" anxiety, the uncertainty of whether a spot will be available upon arrival. By providing real-time data on charger compatibility for EV owners and guaranteed accessibility information for users with

reduced mobility, we aim to make city navigation more inclusive. Additionally, we are driven to give parking managers better tools to monitor their infrastructure, moving away from outdated, non-data-driven management methods.

Use Case Diagram



Personas

Persona #1: The regular driver

Name: Filipe Teixeira

Age: 25

Location: Coimbra, Portugal

Occupation: Economist

Description: Filipe is a young economist from Coimbra who has to frequently drive to Lisbon for weekly meetings with clients and partners. For Filipe, time is his most valuable resource, and he views parking not just as a necessity but as a logistical challenge that needs to be optimized to ensure his punctuality and professional image. If he feels that he might not be able to get a spot on time, he can make a reservation within 30 minutes before arriving at the parking lot.

Motivation: Wants to minimize search time. He needs to check parking availability in advance to avoid driving in circles, ensuring he arrives at his meetings exactly on time without the stress of being late.

- Appreciates price-to-distance ratios: As an economist, he naturally compares costs. He wants to find the best balance between a low-cost spot and its proximity to his specific meeting location.
- Needs transparent and real-time data: He is frustrated by confusing rate structures and outdated information. He expects the app to show the exact price and a live status of the spots to make quick, data-driven decisions.

Persona #2 : The EV (Electric Vehicle) Driver

Name: Maria Silva

Age: 30

Location: Torres Vedras, Portugal

Occupation: Veterinarian

Description: Maria is a veterinarian living in Torres Vedras who commutes daily to her clinic in the city center. Driving a Tesla, her commute isn't just about the 30-minute journey, but also about ensuring her vehicle remains functional for her busy schedule. For Maria, a parking spot must be more than just a space; it must be a reliable service that supports her commitment to electric mobility while fitting into her demanding professional routine.

Motivation:

- Seeks guaranteed charging infrastructure: Maria's primary motivation is to eliminate her anxiety by finding parking spots equipped with functional EV chargers. She needs to ensure her car is charging while she works so it is ready for her return trip.
- Values punctuality and proximity: As a veterinarian with a tight schedule, she is motivated to find sports close to her office. This avoids long walks and ensures she arrives on time for her job, including during peak hours.
- Demands technical transparency: She is frustrated by the lack of specific information regarding charger types, charging speed, prices, and real-time availability. She expects the app to provide live status updates on chargers to avoid arriving at a station only to find it occupied or out of service, allowing her to make efficient, stress-free decisions.

Persona #3 : The Mobility-Impaired Driver

Name: Luís Pedro

Age: 70

Location: Faro, Portugal

Occupation: Retired

Description: Luís has reduced mobility and uses a wheelchair. He frequently visits clinics and hospitals and needs to ensure that his parking spot is not only available but also practical for his specific physical needs.

Motivation: To maintain independence by finding parking spots that are close to his destination (medical services) and that provide enough space to maneuver his wheelchair safely.

- Appreciates: Accurate real-time information about accessible spots, clear walking (or rolling) distances to entrances, and an easy-to-use interface with high contrast.
- Needs: A way to filter specifically for "Accessible/Disabled" spots; To see the exact distance between the spot and the building entrance; Information on whether spots are being monitored to prevent unauthorized use; Assurance that the spot has the necessary lateral space for a wheelchair ramp or exit.

Persona #4 : The Parking Manager

Name: António Videira

Age: 50

Location: Leiria, Portugal

Occupation: Parking Manager

Description: António manages several parking sites, including the one integrated with EasySpot. He is responsible for day-to-day operations, revenue tracking, and ensuring good customer experience across all lots.
Motivation: Keep occupancy high and predictable while maximising revenue and minimising operational issues. António needs clear, timely information to justify decisions and report performance to stakeholders.

- Appreciates: Actionable key performance indicators (daily entries, occupancy rate, turnover), clear revenue figures and exportable reports, and fast alerts for issues that impact availability or customer satisfaction.
- Needs: A read-only dashboard showing daily entries, occupancy by zone and revenue summaries; visibility of fixed tariffs and historical billing for reconciliation; and a consolidated feed of customer reports and sensor alerts with full details to prioritise actions.

Persona #5 : The Maintenance Technician

Name: Laura Farias

Age: 29

Location: Alvalade, Portugal

Occupation: Maintenance Technician

Description: Laura Farias installs and maintains sensors and gateways for parking sites and expects full access to reported issues, sensor metadata and operational logs so she can diagnose faults and plan interventions.

Motivation: Laura wants complete, timely and precise information about sensor faults and customer reports so she can identify root causes remotely, prioritise repairs efficiently and minimise repeat visits and downtime.

- Appreciates: actionable key performance indicators (sensor uptime, false-positive rate, mean time to repair), detailed diagnostic logs and exportable reports, and immediate contextual alerts (sensor ID,last reading).
- Needs: a consolidated problems feed with sensor metadata; remote diagnostic tools (sensor's status); and the ability to create/assign prioritized maintenance orders with technical notes.

Scenarios

Scenario #1: Filipe Teixeira (regular driver)

Filipe is on his way to his weekly meeting in Lisbon with his clients. Unfortunately, due to unexpected traffic on the bridge, he is running late. He is worried that the time spent searching for a parking spot will further delay his meeting and damage his professional image. During the drive, he opens the EasySpot app to check the number of free spots and current rates at the parking lot, so he knows exactly what to look for when he arrives and how much it will cost him. If he feels that he might not be able to get a spot on time he can make a reservation within 30 minutes before arriving at the parking lot.

Scenario #2: Maria Silva (EV driver)

It is a Tuesday morning, and Maria is preparing for her 30-minute commute from Torres Vedras to her veterinary clinic. Her Tesla's battery is lower than usual, and she knows that finding a spot with a working charger near her office is crucial to avoid stress during her return trip.

Before leaving home, Maria opens the EasySpot app. The app shows her a real-time map with a spot that includes a fast-charger compatible with her vehicle. She can see the current charging rates and the estimated occupancy for the next hour.

Scenario #3: Luís Pedro (driver with reduced mobility)

Luís has a follow-up medical appointment at a clinic. As a wheelchair user, he is often anxious about parking because standard spots do not provide enough space for him to exit his vehicle safely, and accessible spots are frequently occupied by unauthorized drivers.

Before leaving home, Luís uses the EasySpot app to filter for available accessible parking spaces. By checking the app, he can see the exact distance he will need to travel in his wheelchair to the entrance and confirm that the spots are being monitored, giving him the peace of mind that he will be able to attend his appointment on time and without physical strain.

Scenario #4: António Videira (Parking Manager)

António manages a network of parking lots and feels constantly frustrated by the lack of concrete data regarding his operations. The systems he currently uses are limited, offering superficial statistics that do not allow him to understand the true financial performance or the actual flow of customers in his assets.

Before starting his strategy meetings, António uses the EasySpot dashboard to gain a clear view of the business. By accessing the platform, he can monitor the exact number of customers who entered the lot daily and cross-reference that data with financial returns in real-time. Additionally, he validates the current tariffs applied and stays informed about all issues, ensuring that the operation runs flawlessly and that the investment is being profitable.

Scenario #5: Laura Farias (Maintenance Technician)

Laura is responsible for ensuring that the entire technological infrastructure of the park works perfectly. In her daily routine, she frequently faces the problem of receiving breakdown alerts that are too generic, which forces her to travel to the site without the right tools or without knowing exactly which sensor or device is failing.

Through EasySpot, Laura has access to a detailed technical panel where all reported problems appear with their specificities given by the users. She can consult the status of each sensor and the error history before even leaving the office.

Product requirements (User stories)

User Story #1: Real-time Parking Availability (High Priority)

As a regular driver,
I want to check the number of available parking spots in advance, so that I can minimize my search time and ensure I arrive at my meetings on time.

Given I am using the EasySpot app.
When I open the parking map
Then I should see the number of available spots updated in real time

User Story #2: Price Transparency & Planning (Low Priority)

As a regular driver,
I want to view the parking lot's rate structures and occupancy statistics, so that I can choose the most cost-effective timing for my stay and manage my expenses.

Given I am using the EasySpot app
When I open the pricing and occupancy section
Then I should see the hourly rates, tariffs, and historical occupancy trends

User Story #3: Parking Spot Reservation (Medium Priority)

As a regular driver,
I want to be able to make a reservation within the time limit (30 minutes before arriving)

Given I am driving to the parking lot
When I open the app on the number of available spots
Then I should be able to reserve a spot
And I should see how much it costs

User Story #4: Total of expenses spent on the parking lot (Low Priority)

As a regular driver,
I want to be able to see how much I have spent on the parking lot

Given I am a driver who parks at the parking lot
When I open the app on my expenses
Then I should see the total of my expenses

User Story #5: Report an unauthorized use of a parking spot (Low Priority)

As a client,
I want to be able to fill in a report if I see an unauthorized parking.

Given I am at a parking lot
When I select the option to report an issue
Then I should be able to submit a report

User Story #6: Real-time Charger Status (High Priority)

As an EV driver,
I want to see the real-time status and specification of EV chargers (speed, connector type), so that I can ensure my Tesla is compatible and a spot is available before I arrive.

Given I am searching for EV charging spots
When I open the EasySpot app and filter by EV spots
Then I should see its connector type, charging speed, and current status

User Story #7: Combined Parking + Charging Fee Overview (Low Priority)

As an EV driver,
I want to view combined parking and charging fees so that I can manage my daily commuting expenses and pay everything in a single transaction.

Given I am viewing an EV-enabled parking spot
When I open the pricing details
Then I should see the total estimated cost (parking + charging)

User Story #8: Accessible Spot Filtering & Proximity (High Priority)

As Mobility-Impaired Driver,
I want to filter the parking spots for the specifically designated accessible and see the exact distance to the parking entrance, so that I can ensure the walk the least distance to my medical appointment is manageable and within my physical limits.

Given I am using the app
When I apply the “Accessible Spots” filter
Then I should only see designated accessible spots
And I should see the distance to the parking entrance

User Story #9: Reliability & Physical Compliance (High Priority)

As a Mobility-Impaired driver,
I want to receive real-time confirmation of spot availability and space dimensions, so that I don't risk arriving at a spot that is either illegally occupied by an unauthorized vehicle or too narrow for me to safely deploy my wheelchair.

Given I am viewing an accessible spot
When I open its details
Then I should see its width, lateral clearance, and real-time occupancy

User Story #10: Real-Time Occupancy & Financial Performance (Low Priority)

As a parking operations manager,
I want to access a dashboard that shows the daily number of customers and real-time financial returns, so that I can evaluate the park’s profitability and make data-driven decisions to optimize my investment.

Given I am logged into the operator dashboard
When I access the analytics section
Then I should see daily entries, occupancy rate, and revenue in real time

User Story #11: Tariff Management & Issue Oversight (Low Priority)

As a parking operations manager,
I want to view the current tariff structures and a centralized log of all problems, so that I can ensure the pricing strategy is correct and maintain a high standard of service quality across my facilities.

Given I am in the management dashboard
When I open the tariffs section
Then I should see all current tariffs and their effective dates
And when I open the issues log
Then I should see all reported problems with timestamps and categories

User Story #12: Technical Specificity & Remote Diagnostics (Low Priority)

As a field maintenance technician,
Accessing sensor logs and connection data will allow me to troubleshoot issues remotely and be fully prepared before traveling to the site.

Given I am viewing the maintenance panel
When I select a reported issue
Then I should see the sensor ID
And I should see the full error history

User Story #13: Infrastructure Mapping & Status Monitoring (High Priority)

As a field maintenance technician,
I want to see a live map of all hardware components and their individual status, so that I can perform proactive maintenance and ensure the parking ecosystem remains fully operational for all users.

Given I am in the technical dashboard
When I open the infrastructure map
Then I should see all sensors and their information
And each component should display its real-time operational status

User Story #14: Repair & Update of sensors (High Priority)

As a field maintenance technician,
I want to be able to update the state of the sensors when I repair a malfunction. So that the system accurately reflects their current operational condition.

Given I am on the technical dashboard
When I repair a sensor malfunction
Then I should be able to update the sensor’s status
And the error statistics should be updated accordingly to reflect the change.

3. Architecture notebook

Key quality requirements

Non-functional requirements and constraints

As a real-time parking analytics application, the primary focus of **EasySpot** is to enable a system that has a low-latency data flow for real-time occupancy events and a persistent and scalable database that stores historical and statistical data. With this, the user can interact with live parking maps and see availability for regular, EV, and accessible spots. Another important requirement of our application is to have a user-friendly and responsive interface that prioritizes ease of use. This is to allow drivers, who may be in a rush, to access the information they need without much trouble.

Performance and Responsiveness - As a real-time monitoring application, it is important to have low latency; for example, when a car leaves a parking spot, the user should see that spot marked as available on the platform in under 5 seconds.

Scalability - EasySpot needs to be able to keep working even if the traffic of users or the number of connected sensors increases. The application must be able to handle hundreds of simultaneous sensor updates and up to 500 to 1000 users without noticeable performance degradation.

Data Consistency and Integrity - Being a parking management app, we need to make sure that the data and statistics shown to the user are updated and accurate. It is critical that the system correctly identifies the type of spot (EV vs. Regular) and reflects its actual state to avoid misleading drivers.

Deployment and Portability (Constraints) - The system is constrained by the requirement to be fully containerized using Docker and Docker Compose. This ensures that the entire environment, including the database and data simulation pipelines, can be deployed consistently across different servers.

Functional requirements and constraints

The main functionalities of **EasySpot** revolve around providing real-time occupancy data for various types of parking spots, historical usage analytics for managers, and a simulation environment for sensor data. To ensure a seamless experience for different users, the platform integrates specialized filters for EV charging and accessibility, along with a notification system for critical parking events or technical malfunctions.

Real-time Monitoring & Alerts - This system will be able to alert the user of events that are happening in the parking lots they are interested in, such as when a lot reaches full capacity or when a specific spot type (like an EV charger) becomes available. It also alerts maintenance technicians if a sensor stops reporting data or has a malfunction.

Parking Simulation - The simulation generates a realistic stream of occupancy events based on historical patterns. This allows us to test the **Reservation System** and the data pipeline without needing physical sensors.

Architectural view

For an easier understanding of how EasySpot is going to work we divided the architecture into 4 parts, those being frontend, backend, data sources and message queues.

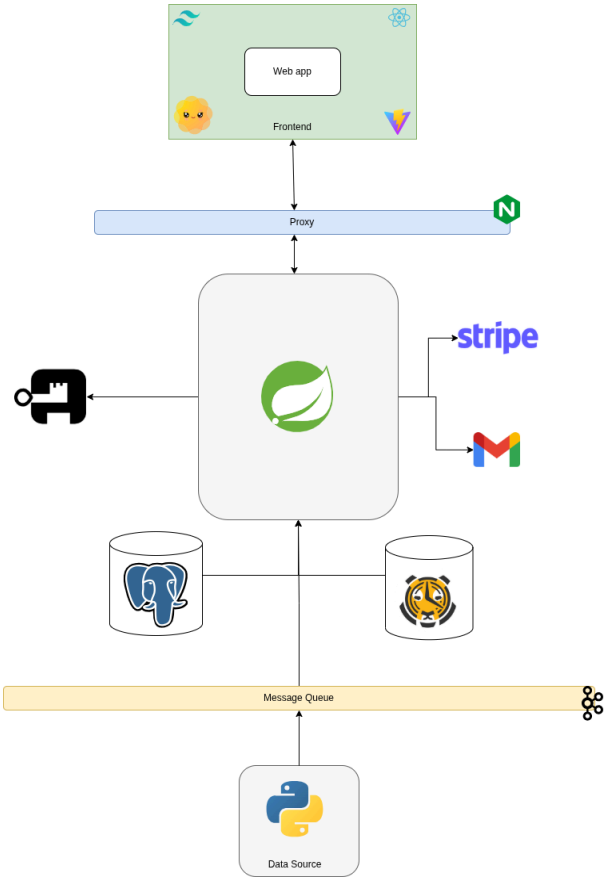
Data Sources: The foundation of the system is the Data Source Layer, which utilizes a Python-based simulation environment that functions as a comprehensive network of virtual IoT sensors. This layer is responsible for generating a continuous stream of real-time events that mimic physical parking infrastructure, including changes in spot occupancy, electric vehicle charging levels, and the operational health status of hardware components across various urban sites.

Message Queues: This Layer employs Apache Kafka to serve as a high-throughput message broker. This layer effectively decouples data generation from the processing stages, ensuring that the system can handle thousands of simultaneous sensor updates asynchronously. This architectural choice prevents backend bottlenecks and guarantees that the data pipeline remains resilient even during peak traffic periods.

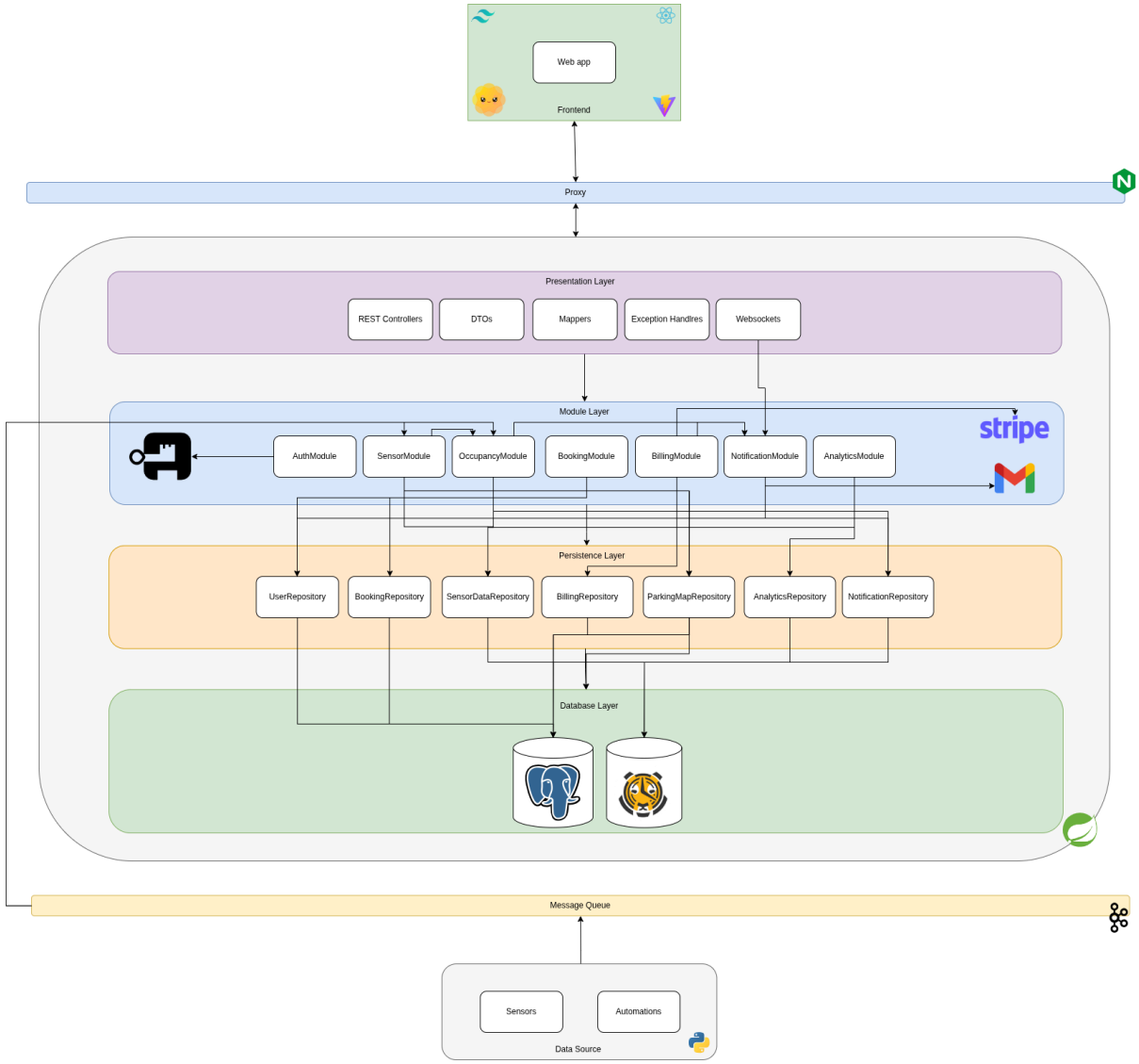
Backend: The Backend Layer, developed using the Spring Boot framework, serves as the core intelligence of the application by managing data processing, business logic, and complex external integrations. It implements a strategy to optimize data management, utilizing PostgreSQL for relational data such as user accounts and reservations, while leveraging TimescaleDB for time-series data like occupancy trends and sensor logs. Additionally, this layer facilitates critical external services through the Stripe API for secure payment processing and the Gmail SMTP protocol for automated user notifications.

Frontend: The Frontend Layer provides the primary interface for users through a React Web App powered by Vite, designed for high performance and responsiveness. This layer interacts with the backend via REST APIs for standard transactional operations like searching and booking, while simultaneously maintaining Websocket connections to deliver instantaneous updates to the live parking map, ensuring that drivers receive the most accurate availability data without manual refreshing.

Base Architecture



Layered Architecture



Module interactions

The EasySpot system consists of interconnected modules that communicate with each other to provide a robust and scalable environment. The key interactions between modules are as follows:

1. **Virtual Sensors and Kafta:**
   Virtual sensors are simulated by Python agents that generate real-time occupancy events such as vehicle entry and exit, EV charger changes, and accessible spot availability updates. These events are published to Kafta topics, which acts as a message broker, decoupling data production from its processing.

2. **Spring Boot Middleware and databases:**
   The Spring Boot backend consumes Kafka topics through the SensorModule, which validates and normalises incoming data before forwarding it to the OccupancyModule. The OccupancyModule is responsible for updating the real-time occupancy state of each parking spot and persisting it to the SensorDataRepository and ParkingMapRepository. When a spot's state changes, the OccupancyModule also checks whether the lot has reached full capacity, triggering a notification event if applicable.
3. **Booking Module**
   The Booking Module handles spot reservation requests received via REST Controllers. Before confirming a reservation, it queries the OccupancyModule to verify real-time availability and temporarily locks the spot for up to 30 minutes. It then communicates with the BillingModule to calculate the estimated cost of the session. Once confirmed, the booking record is persisted in the BookingRepository.
4. **Billing Module**
   The Booking Module integrates with the Stripe payment gateway to process payments. It receives session events from the Booking Module, such as reservation start and session end, and generates billing records stored in the Billing Repository. After a successful payment intent is created, it communicates with the BillingRepository. After a successful attempt with the payment intent is created, it communicates with the NotificationModule to dispatch payment confirmation messages to the user.
5. **Notification Module**
   The Notification Module subscribes to events from multiple modules, including full-lot-capacity events from the Occupancy Module, sensor-fault events from the Sensor Module, and booking confirmations from the Billing Module. It builds alert payloads and delivers them through two channels: real-time alerts are pushed via WebSockets to connected clients, while email notifications are dispatched through Gmail's SMTP service, used for booking confirmations, payment receipts, and critical sensor fault alerts directed at maintenance technicians. All notification history is persisted in the NotificationRepository, enabling technicians and managers to consult past alerts.
6. **Analytics Module**
   The Analytics Module aggregates data from the Occupancy Module and Billing Module to generate operational performance reports, including daily entries, occupancy rate, revenue summaries, and tariff history. This data is exposed through REST Controllers to the operator dashboard used by parking managers. Aggregated records are stored in the AnalyticsRepository.

7. **Auth Module**
   The Auth Module manages authentication and access control using JWT tokens, delegating identity management to Authentik. Authentik handles user login flows, session management, and role assignment, issuing signed JWT tokens that the AuthModule validates on every incoming request. This ensures that only authorised users can access role-specific endpoints, drivers, managers, and technicians each of whom has distinct permission scopes enforced at the API gateway level. User data is persisted in the UserRepository.

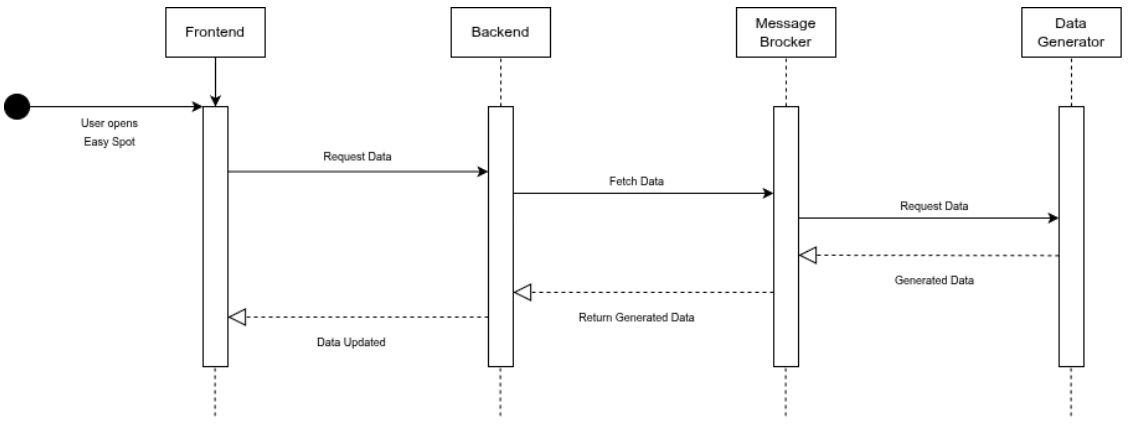
8. **Frontend, Proxy and Presentation Layer**
   The web and mobile clients consume REST endpoints and WebSocket streams exposed by the Spring Boot Presentation Layer, which includes REST Controllers, DTOs, Mappers, Exception Handlers, and WebSocket handlers. All traffic is routed through a reverse proxy that centralises ingress and enables load balancing. WebSockets are used exclusively for real-time updates such as live spot availability on the parking map and sensor fault alerts on the technician dashboard.

Sequence diagrams

The following diagrams describe the behaviour of the EasySpot system across four key scenarios.

User Opens EasySpot (Real-Time Data Load)
This diagram describes the initial data load flow triggered when a user opens the EasySpot application.

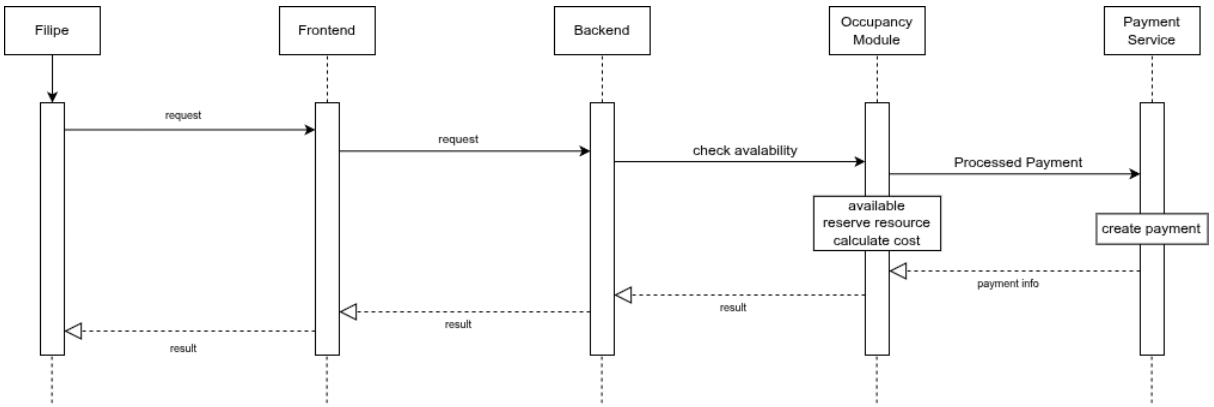
- The Frontend sends a data request to the Backend, which fetches live occupancy data from the Message Broker.
- The Message Broker requests the latest data from the Data Generator, which produces and returns the generated sensor readings.
- The Backend processes the response and returns it to the Frontend, which updates the interface with the current parking availability.



Spot Reservation (User Story #3)

This diagram covers the reservation flow triggered by a regular driver such as Filipe, who needs to secure a parking spot up to 30 minutes before arriving.

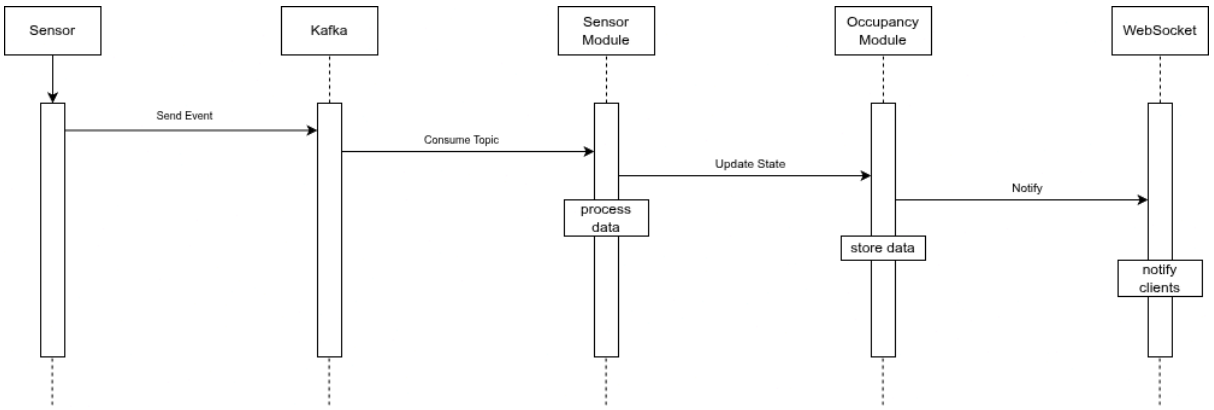
- The flow starts when the user submits a reservation request through the Frontend, which forwards it to the Backend.
- The Backend verifies real-time spot availability and temporarily locks the spot for 30 minutes. At this point the flow branches: if the spot is available, the Backend proceeds to calculate the estimated cost and initiates a payment intent via the external billing integration; if unavailable, an error response is returned to the Frontend and the flow terminates.
- Once confirmed, the booking record is persisted and a confirmation response is returned to the Frontend, which displays the reserved spot and its cost to the user.



Real-Time Occupancy Update (User Story #1)

This diagram describes when a sensor detects a change in a parking spot's state and the update is propagated in real time to all connected clients.

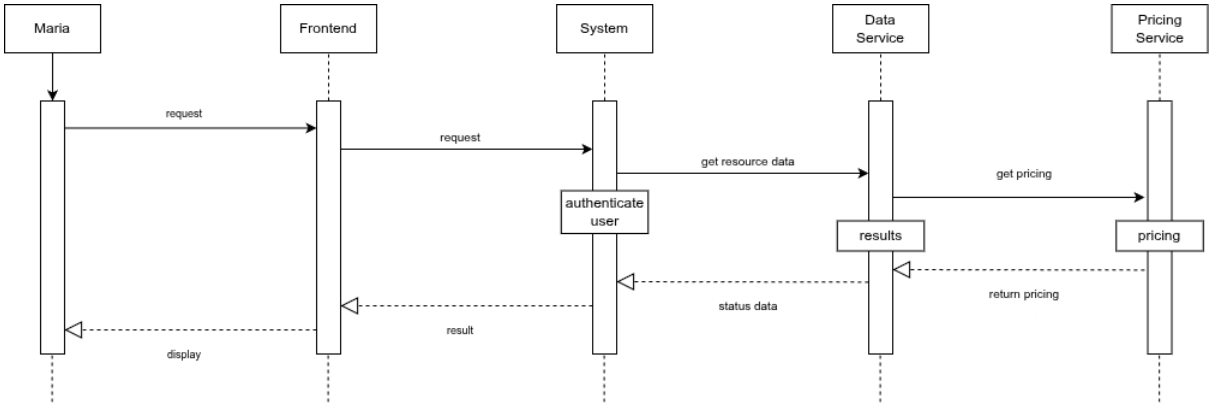
- The flow begins when a Sensor publishes an occupancy event to Kafka.
- The Sensor Module consumes the topic and processes the incoming data before forwarding it to the Occupancy Module, which updates the spot's state and stores it persistently.
- The updated state is then pushed via WebSocket to all connected Frontend clients, ensuring drivers see live availability within the system's 5-second latency requirement.



EV Charger Status (User Story #6)

This diagram describes the flow triggered when an EV driver such as Maria searches for an available charging spot compatible with her vehicle.

- The flow begins when Maria submits a request through the Frontend.
- The request reaches the System, which first authenticates the user before proceeding.
- Once authorised, the System queries the Data Service to retrieve live charger availability and spot details, and simultaneously queries the Pricing Service to obtain the applicable parking and charging rates.
- The Data Service and Pricing Service return their results to the System, which assembles the full response and returns it to the Frontend.
- The Frontend then displays a live map showing only available EV spots with their technical specifications and combined cost estimate.



Sensor Fault Alert (User Story #13)

This diagram describes the fully asynchronous fault detection flow, which is triggered automatically when a sensor stops reporting data, without any user action.

The flow begins when an Event Source delivers a fault event to the System.

- The System detects the issue, retrieves the latest available data from the Data Store to confirm the fault, and updates the sensor's status accordingly.
- It then creates an alert and triggers the Notification Service, which notifies Laura directly on her technician dashboard.
- Laura views the alert, consults the error details remotely, and once the repair is complete, sends an update status response back to the System, which clears the fault and restores accurate availability data for the affected spot.